

Escuela Politécnica Nacional

Nombre: Kevin Eduardo Garcia Rodriguez

Tema: [Taller 02b] Descomposicion LU- Matriz Inversa

Repositorio GIT: <https://github.com/Nattyrd/Metodos-Numericos-2025B>

```
In [2]: import numpy as np
import logging
```

```
In [3]: from src import (
    eliminacion_gaussiana,
    descomposicion_LU,
    resolver_LU,
    matriz_aumentada,
    separar_m_aumentada,

)
```

[01-13 23:19:41][INFO] 2026-01-13 23:19:41.386962

[01-13 23:19:41][INFO] 2026-01-13 23:19:41.392564

EJERCICIO 1-Obtener la inversa de la matriz.

```
In [7]: def Jordan_gaussiana(A: np.ndarray) -> np.ndarray:
    """
    Aplica el método de Gauss-Jordan normalizado y devuelve
    la matriz en forma reducida por filas (RREF).
    """

    if not isinstance(A, np.ndarray):
        A = np.array(A, dtype=float)

    n, m = A.shape
    assert m >= n, "La matriz debe ser aumentada."

    for i in range(n):
        # --- buscar pivote
        p = None
        for pi in range(i, n):
            if A[pi, i] != 0:
                p = pi
                break

        if p is None:
            raise ValueError("No existe solución única.")

        # --- intercambio de filas
        if p != i:
            A[[i, p]] = A[[p, i]]

        # --- normalizar pivote
        pivote = A[i, i]
        A[i, :] = A[i, :] / pivote
```

```

    # --- eliminar arriba y abajo
    for j in range(n):
        if j != i:
            factor = A[j, i]
            A[j, :] = A[j, :] - factor * A[i, :]

    return A

```

```

In [8]: def inversa_por_jordan_normalizado(A: np.ndarray) -> np.ndarray:
        n = A.shape[0]
        AI = np.hstack((A, np.eye(n))) ## Crear matriz aumentada [A | I] Horizontalm
        R = Jordan_gaussiana(AI)

        A_inv = R[:, n:]
        return A_inv

```

Matriz 1

```

In [73]: A01 = np.array([
        [1, 3, 4],
        [2, 1, 3],
        [4, 2, 1]
    ])

```

```

In [74]: A01_inv=inversa_por_jordan_normalizado(A01)
        print("La inversa de A01 es:\n", A01_inv)

```

La inversa de A01 es:

```

[[-0.2  0.2  0.2]
 [ 0.4 -0.6  0.2]
 [-0.   0.4 -0.2]]

```

Comprobacion con libreria numpy

```

In [ ]: np.linalg.inv(np.array(A01)) ##Respuesta correcta

```

```

Out[ ]: array([[ -0.2,  0.2,  0.2],
               [ 0.4, -0.6,  0.2],
               [ 0. ,  0.4, -0.2]])

```

Matriz 2

```

In [76]: A02 = np.array([
        [1, 2, 3],
        [0, 1, 4],
        [5, 6, 0]
    ])

```

```

In [80]: A02_inv=inversa_por_jordan_normalizado(A02)
        print("La inversa de A02 es:\n", A02_inv)

```

La inversa de A02 es:

```

[[-24.  18.   5.]
 [ 20. -15.  -4.]
 [ -5.   4.   1.]]

```

Matriz 3

```
In [78]: A03 = np.array([
    [4, 2, 1],
    [2, 1, 3],
    [1, 3, 4]
])
```

```
In [81]: A03_inv=inversa_por_jordan_normalizado(A03)
print("La inversa de A03 es:\n", A03_inv)
```

La inversa de A03 es:

```
[[ 0.2  0.2 -0.2]
 [ 0.2 -0.6  0.4]
 [-0.2  0.4  0. ]]
```

Matriz 4

```
In [82]: A04 = np.array([
    [2, 4, 6, 1],
    [4, 7, 5, -6],
    [2, 5, 18, 10],
    [6, 12, 38, 16]
])
```

```
In [83]: A04_inv=inversa_por_jordan_normalizado(A04)
print("La inversa de A04 es:\n", A04_inv)
```

La inversa de A04 es:

```
[[ 3.2         -1.71111111 -3.71111111  1.47777778]
 [-0.4         0.53333333  1.53333333 -0.73333333]
 [-0.8         0.28888889  0.28888889 -0.02222222]
 [ 1.          -0.44444444 -0.44444444  0.11111111]]
```

EJERCICIO 2-Obtener la descomposicion LU para las siguientes matrices y calcular con los vectores dados.

```
In [9]: A1 = np.array([
    [1, 3, 4],
    [2, 1, 3],
    [4, 2, 1]
])

L1,U1=descomposicion_LU(A1)
```

```
[01-13 23:09:41][INFO]
[[ 1.  3.  4.]
 [ 0. -5. -5.]
 [ 0. -10. -15.]]
[01-13 23:09:41][INFO]
[[ 1.  3.  4.]
 [ 0. -5. -5.]
 [ 0.  0. -5.]]
[01-13 23:09:41][INFO]
[[ 1.  3.  4.]
 [ 0. -5. -5.]
 [ 0.  0. -5.]]
```

```
In [10]: b= np.array([1, 2, 4])
x, y = resolver_LU(L1, U1, b)

print("Vector y:")
print(y)

print("Solución x:")
print(x)
```

```
[01-13 23:09:44][INFO] Sustitución hacia adelante
[01-13 23:09:44][INFO] y[0] = [1.]
[01-13 23:09:44][INFO] y =
[[1.]
 [0.]
 [0.]]
[01-13 23:09:44][INFO] Sustitución hacia atrás
[01-13 23:09:44][INFO] i = 1
[01-13 23:09:44][INFO] suma = [0.]
[01-13 23:09:44][INFO] U[i, i] = -5.0
[01-13 23:09:44][INFO] y[i] = [0.]
[01-13 23:09:44][INFO] i = 0
[01-13 23:09:44][INFO] suma = [0.]
[01-13 23:09:44][INFO] U[i, i] = 1.0
[01-13 23:09:44][INFO] y[i] = [1.]
Vector y:
[[1.]
 [0.]
 [0.]]
Solución x:
[[ 1.]
 [-0.]
 [-0.]]
```

```
In [8]: A2 = np.array([
    [1, 2, 3],
    [0, 1, 4],
    [5, 6, 0]
])
L2,U2=descomposicion_LU(A2)
```

```
[01-13 23:24:28][INFO]
[[ 1.  2.  3.]
 [ 0.  1.  4.]
 [ 0. -4. -15.]]
[01-13 23:24:28][INFO]
[[1. 2. 3.]
 [0. 1. 4.]
 [0. 0. 1.]]
[01-13 23:24:28][INFO]
[[1. 2. 3.]
 [0. 1. 4.]
 [0. 0. 1.]]
[01-13 23:24:28][INFO]
[[1. 2. 3.]
 [0. 1. 4.]
 [0. 0. 1.]]
```

```
In [9]: b= np.array([3, -5, 2])
x, y = resolver_LU(L2, U2, b)
```

```
b11= np.array([7, 8, -1])
x11, y11 = resolver_LU(L2, U2, b11)

print("Vector y:")
print(y)

print("Solución x:")
print(x)

print("Vector y11:")
print(y11)
print("Solución x11:")
print(x11)
```

```

[01-13 23:24:30][INFO] Sustitución hacia adelante
[01-13 23:24:30][INFO] y[0] = [3.]
[01-13 23:24:30][INFO] y[0] = [3.]
[01-13 23:24:30][INFO] y =
[[ 3.]
 [ -5.]
 [-33.]]
[01-13 23:24:30][INFO] Sustitución hacia atrás
[01-13 23:24:30][INFO] i = 1
[01-13 23:24:30][INFO] suma = [-132.]
[01-13 23:24:30][INFO] U[i, i] = 1.0
[01-13 23:24:30][INFO] y[i] = [-5.]
[01-13 23:24:30][INFO] i = 0
[01-13 23:24:30][INFO] suma = [155.]
[01-13 23:24:30][INFO] U[i, i] = 1.0
[01-13 23:24:30][INFO] y[i] = [3.]
[01-13 23:24:30][INFO] Sustitución hacia adelante
[01-13 23:24:30][INFO] y[0] = [7.]
[01-13 23:24:30][INFO] y =
[[ 7.]
 [ 8.]
 [-4.]]
[01-13 23:24:30][INFO] Sustitución hacia atrás
[01-13 23:24:30][INFO] i = 1
[01-13 23:24:30][INFO] suma = [-16.]
[01-13 23:24:30][INFO] U[i, i] = 1.0
[01-13 23:24:30][INFO] y[i] = [8.]
[01-13 23:24:30][INFO] i = 0
[01-13 23:24:30][INFO] suma = [36.]
[01-13 23:24:30][INFO] U[i, i] = 1.0
[01-13 23:24:30][INFO] y[i] = [7.]
Vector y:
[[ 3.]
 [ -5.]
 [-33.]]
Solución x:
[[-152.]
 [ 127.]
 [ -33.]]
Vector y11:
[[ 7.]
 [ 8.]
 [-4.]]
Solución x11:
[[-29.]
 [ 24.]
 [ -4.]]

```

```

In [ ]: A3 = np.array([
    [4, 2, 1],
    [2, 1, 3],
    [1, 3, 4]
])
descomposicion_LU(A3) ##NO SE PUEDE APLICAR LU (Requiere intercambio de filas)

```

```

[01-13 16:32:53][INFO]
[[4.  2.  1. ]
 [0.  0.  2.5 ]
 [0.  2.5  3.75]]

```

```

-----
ValueError                                Traceback (most recent call last)
Cell In[8], line 6
      1 A3 = np.array([
      2     [4, 2, 1],
      3     [2, 1, 3],
      4     [1, 3, 4]
      5 ])
----> 6 descomposicion_LU(A3)

File c:\Users\NattyrdGT\Documents\Codigos\Metodos2B\src\linear_syst_methods.py:12
7, in descomposicion_LU(A)
    123 for i in range(0, n): # loop por columna
    124
    125     # --- deterimnar pivote
    126     if A[i, i] == 0:
--> 127         raise ValueError("No existe solución única.")
    129     # --- Eliminación: loop por fila
    130     L[i, i] = 1

ValueError: No existe solución única.

```

```

In [4]: A4 = np.array([
        [2, 4, 6, 1],
        [4, 7, 5, -6],
        [2, 5, 18, 10],
        [6, 12, 38, 16]
    ])
    L4,U4=descomposicion_LU(A4)

```

```

[01-13 23:19:46][INFO]
[[ 2.  4.  6.  1.]
 [ 0. -1. -7. -8.]
 [ 0.  1. 12.  9.]
 [ 0.  0. 20. 13.]]
[01-13 23:19:46][INFO]
[[ 2.  4.  6.  1.]
 [ 0. -1. -7. -8.]
 [ 0.  0.  5.  1.]
 [ 0.  0. 20. 13.]]
[01-13 23:19:46][INFO]
[[ 2.  4.  6.  1.]
 [ 0. -1. -7. -8.]
 [ 0.  0.  5.  1.]
 [ 0.  0. 20. 13.]]
[01-13 23:19:46][INFO]
[[ 2.  4.  6.  1.]
 [ 0. -1. -7. -8.]
 [ 0.  0.  5.  1.]
 [ 0.  0.  0.  9.]]
[01-13 23:19:46][INFO]
[[ 2.  4.  6.  1.]
 [ 0. -1. -7. -8.]
 [ 0.  0.  5.  1.]
 [ 0.  0.  0.  9.]]

```

```

In [6]: b1=np.array([1, 2, 4, 5])
        b2=np.array([3, -5, 2, 6])
        b3=np.array([7, 8, -1, 0])

```

```
x1, y1 = resolver_LU(L4, U4, b1)
x2, y2 = resolver_LU(L4, U4, b2)
x3, y3 = resolver_LU(L4, U4, b3)

print("Vector y1:")
print(y1)
print("Solución x1:")
print(x1)

print("Vector y2:")
print(y2)
print("Solución x2:")
print(x2)

print("Vector y3:")
print(y3)
print("Solución x3:")
print(x3)
```



```

[01-13 23:22:42][INFO] Sustitución hacia adelante
[01-13 23:22:42][INFO] y[0] = [1.]
[01-13 23:22:42][INFO] y[0] = [1.]
[01-13 23:22:42][INFO] y =
[[ 1.]
 [ 0.]
 [ 3.]
 [-10.]]
[01-13 23:22:42][INFO] Sustitución hacia atrás
[01-13 23:22:42][INFO] i = 2
[01-13 23:22:42][INFO] suma = [-1.11111111]
[01-13 23:22:42][INFO] U[i, i] = 5.0
[01-13 23:22:42][INFO] y[i] = [3.]
[01-13 23:22:42][INFO] i = 1
[01-13 23:22:42][INFO] suma = [3.13333333]
[01-13 23:22:42][INFO] U[i, i] = -1.0
[01-13 23:22:42][INFO] y[i] = [0.]
[01-13 23:22:42][INFO] i = 0
[01-13 23:22:42][INFO] suma = [16.35555556]
[01-13 23:22:42][INFO] U[i, i] = 2.0
[01-13 23:22:42][INFO] y[i] = [1.]
[01-13 23:22:42][INFO] Sustitución hacia adelante
[01-13 23:22:42][INFO] y[0] = [3.]
[01-13 23:22:42][INFO] y =
[[ 3.]
 [-11.]
 [-12.]
 [ 45.]]
[01-13 23:22:42][INFO] Sustitución hacia atrás
[01-13 23:22:42][INFO] i = 2
[01-13 23:22:42][INFO] suma = [5.]
[01-13 23:22:42][INFO] U[i, i] = 5.0
[01-13 23:22:42][INFO] y[i] = [-12.]
[01-13 23:22:42][INFO] i = 1
[01-13 23:22:42][INFO] suma = [-16.2]
[01-13 23:22:42][INFO] U[i, i] = -1.0
[01-13 23:22:42][INFO] y[i] = [-11.]
[01-13 23:22:42][INFO] i = 0
[01-13 23:22:42][INFO] suma = [-36.2]
[01-13 23:22:42][INFO] U[i, i] = 2.0
[01-13 23:22:42][INFO] y[i] = [3.]
[01-13 23:22:42][INFO] Sustitución hacia adelante
[01-13 23:22:42][INFO] y[0] = [7.]
[01-13 23:22:42][INFO] y =
[[ 7.]
 [-6.]
 [-14.]
 [ 35.]]
[01-13 23:22:42][INFO] Sustitución hacia atrás
[01-13 23:22:42][INFO] i = 2
[01-13 23:22:42][INFO] suma = [3.88888889]
[01-13 23:22:42][INFO] U[i, i] = 5.0
[01-13 23:22:42][INFO] y[i] = [-14.]
[01-13 23:22:42][INFO] i = 1
[01-13 23:22:42][INFO] suma = [-6.06666667]
[01-13 23:22:42][INFO] U[i, i] = -1.0
[01-13 23:22:42][INFO] y[i] = [-6.]
[01-13 23:22:42][INFO] i = 0
[01-13 23:22:42][INFO] suma = [-17.84444444]
[01-13 23:22:42][INFO] U[i, i] = 2.0

```

```
[01-13 23:22:42][INFO] y[i] = [7.]
```

```
Vector y1:
```

```
[[ 1.]  
 [ 0.]  
 [ 3.]  
 [-10.]]
```

```
Solución x1:
```

```
[[ -7.67777778]  
 [ 3.13333333]  
 [ 0.82222222]  
 [-1.11111111]]
```

```
Vector y2:
```

```
[[ 3.]  
 [-11.]  
 [-12.]  
 [ 45.]]
```

```
Solución x2:
```

```
[[19.6]  
 [-5.2]  
 [-3.4]  
 [ 5. ]]
```

```
Vector y3:
```

```
[[ 7.]  
 [-6.]  
 [-14.]  
 [ 35.]]
```

```
Solución x3:
```

```
[[12.42222222]  
 [-0.06666667]  
 [-3.57777778]  
 [ 3.88888889]]
```